53. Capstone: Production-Shaped HTTP API

53. Capstone: Production-Shaped HTTP API

Phase: Phase 8 — Capstones
Estimated time: 3–5 days
Slug: `53-capstone-infra`

Learning objectives

- Ship an API with graceful shutdown
- Add metrics, retries, worker pool
- Write integration tests
- Document runbook notes

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

The infra capstone stitches HTTP, middleware, context, and reliability into one service-shaped project.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Define routes, middleware, and config.
- Use context deadlines on outbound calls.
- Graceful shutdown on signal.
- Tests via httptest and table cases.
- README that explains how to run locally.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-capstone: && cd /tmp/gw-capstone:
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import (
    "fmt"
    "net/http"
    "net/http/httptest"
)

func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("/healthz", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprint(w, "ok")
    })
    rr := httptest.NewRecorder()
    mux.ServeHTTP(rr, httptest.NewRequest(http.MethodGet, "/healthz", nil))
    fmt.Println(rr.Body.String())
}
```

Run it:

```
go run .
```

Expected output:

```
ok
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Wire /healthz and one domain route.
2. Add middleware request IDs.
3. Write httptest coverage for happy + error paths.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Mission

Build a small but production-shaped service:

- REST endpoints for a job queue resource
- Enqueue is fast; workers process asynchronously
- Graceful shutdown drains workers
- Structured logs + basic metrics counters
- Outbound calls use retries + timeouts
- `GET /healthz` and `GET /readyz`

Milestones

1. Skeleton server + config

2. In-memory store + handlers
3. Worker pool + backpressure
4. Observability + shutdown
5. Tests + README

Rubric (excellent)

- No goroutine leaks on shutdown
- Race-clean tests
- Clear package boundaries
- Explicit error types for API
- Timeouts everywhere

Failure modes to watch while coding

- God-object main.go
- In-memory only without interfaces (hard to test)
- Skipping readiness vs liveness

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- Flask app factory ≈ main + NewServer constructor.
- Same need for tests without real ports.
- Deploy as a single static binary — Go advantage.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

Ship vertical slices: one endpoint fully correct beats five stubs.

Practice checklist

Before the exercise, confirm you can:

1. Explain “Capstone: Production-Shaped HTTP API” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- God-object main.go
- In-memory only without interfaces (hard to test)
- Skipping readiness vs liveness

Exercises

Work in `exercises/53-capstone-infra/`. Read that folder's `README.md` before you code.

What you will build

Build the capstone API stubs until integration-style tests pass.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/53-capstone-infra/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. What does readyz mean?
2. How prove no leaks?
3. Where put business logic?

Further reading

- https://go.dev/doc/tutorial/web-service-gin